

中文法律RAG聊天机器人系统说明书

版本: 1.0.0

日期: 2026-06-24

文档类型: 技术系统说明书

目录

- [1. 系统概述](#)
- [2. 系统架构](#)
- [3. 技术栈](#)
- [4. 核心模块详解](#)
- [5. 数据流与处理流程](#)
- [6. API接口规范](#)
- [7. 数据库设计](#)
- [8. 前端架构](#)
- [9. 部署与运维](#)
- [10. 关键设计决策](#)

1. 系统概述

1.1 系统定位

本系统是一个基于检索增强生成（RAG）技术的中文法律智能问答系统。系统采用IRAC法律分析框架（Issue-Rule-Application-Conclusion），能够对用户提出的法律问题进行结构化分析，提供专业的法律意见。

1.2 核心能力

能力	描述
法律问答	支持刑法、民法、行政法等14个法律领域的专业问答
IRAC分析	自动识别法律问题、检索法律依据、进行法律推理、得出结论

混合检索	标签精准匹配 + 向量语义搜索 + 关系扩展的三路检索策略
溯源引用	每个回答附带法律条文和案例的溯源卡片，支持全文查看
多模态支持	支持文件上传（OCR识别）、语音合成（TTS）、语音识别（STT）

1.3 数据规模

- **法律原文:** 26部法律的.docx文件
 - **司法解释:** 9份司法解释的.docx文件
 - **案例库:** 200+案例的.docx文件
 - **向量库:** 约3000+条法律/案例记录的向量化存储
-

2. 系统架构

2.1 整体架构图

用户层 (User Layer)

Next.js 前端应用 (Port 3000)

登录页面 | 注册页面 | 法律助手 | 消息/IRAC渲染

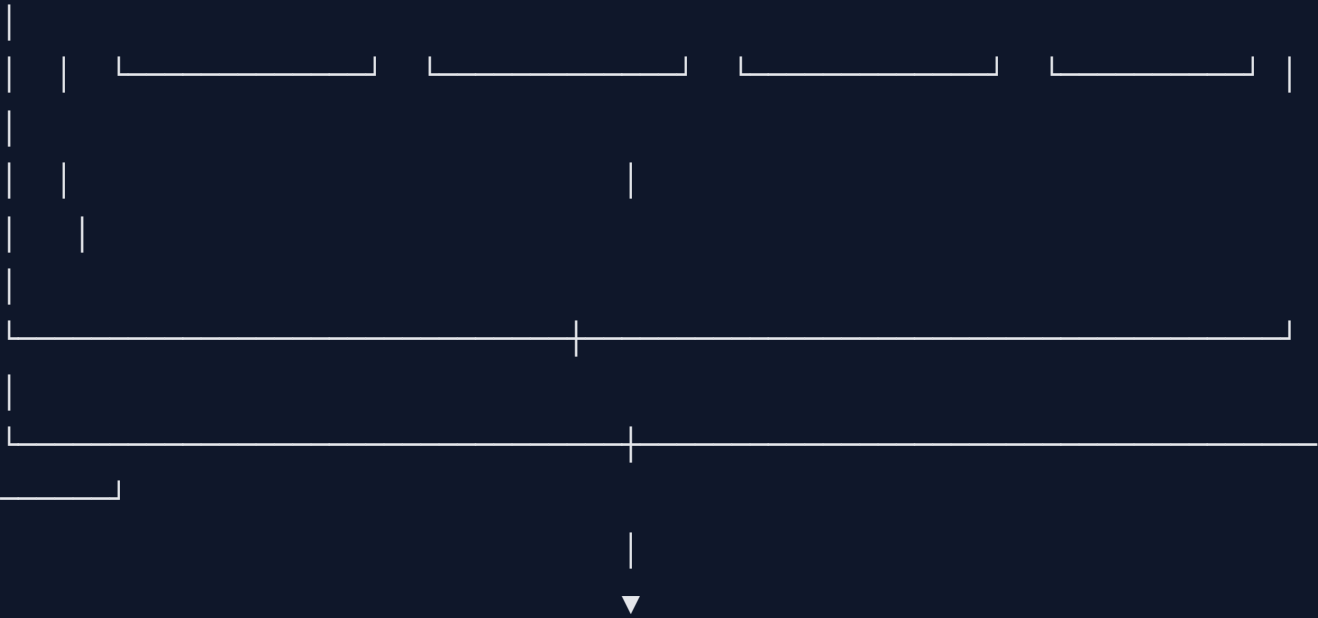
REST API + SSE



服务层 (Service Layer)

FastAPI 后端服务 (Port 8000)

认证模块 | 聊天模块 | 文件上传 | 语音服务
/api/auth/* | /api/chat | /api/upload | /api/tts



核心引擎层 (Core Engine)

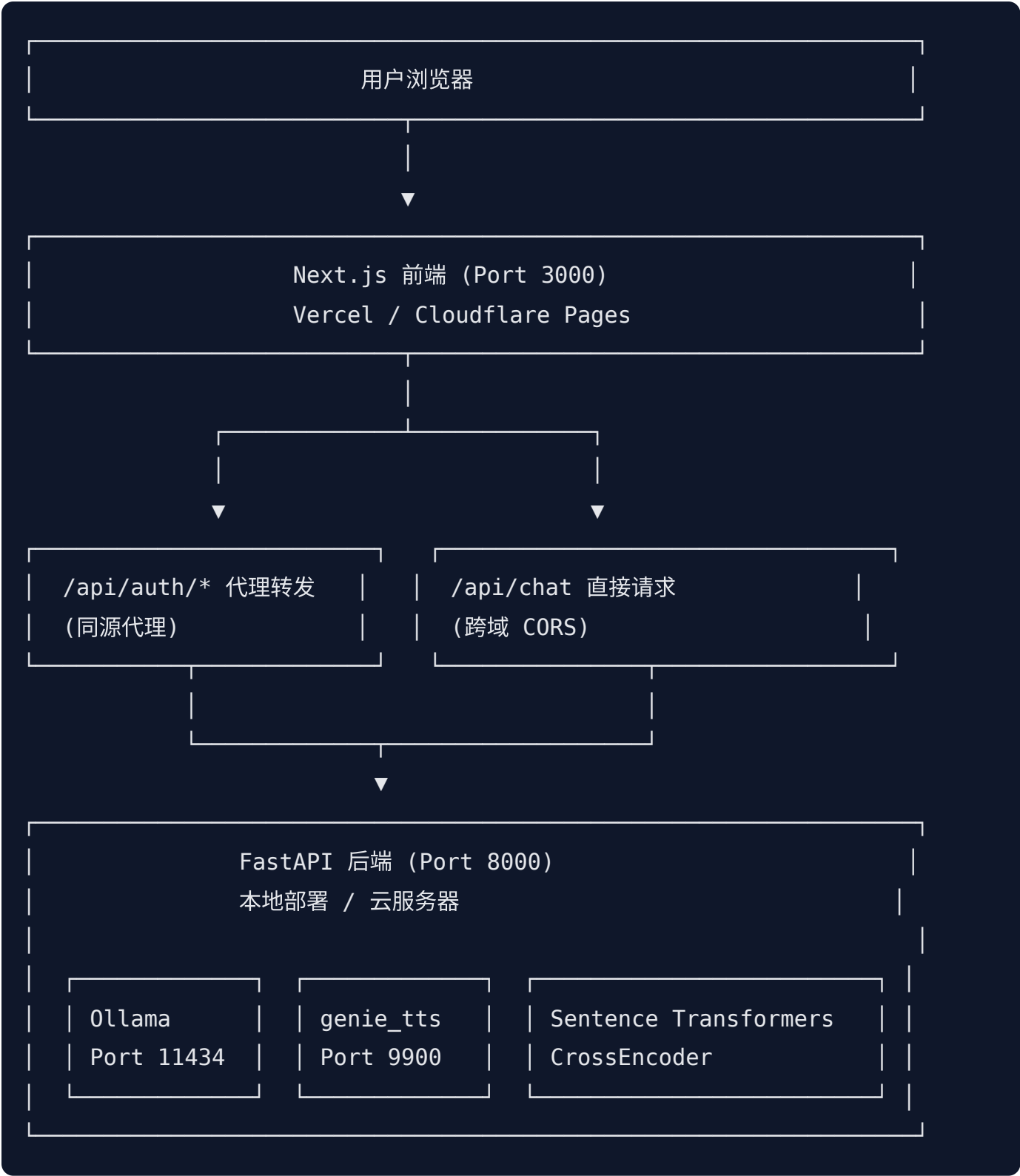
RAG 法律分析流水线



Ollama LLM (Qwen 2.5 7B)



2.2 部署架构



3. 技术栈

3.1 后端技术栈

组件	技术选型	版本	用途
----	------	----	----

Web框架	FastAPI	-	异步HTTP服务
ORM	SQLAlchemy (aiosqlite)	-	异步数据库操作
LLM推理	Ollama	-	本地LLM服务
Embedding	Sentence Transformers	-	文本向量化
Reranker	CrossEncoder (BAAI/bge-reranker-v2-m3)	-	检索结果重排序
密码哈希	passlib (Argon2/bcrypt)	-	用户密码安全存储
JWT	PyJWT	-	身份认证令牌
TTS	genie_tts (ONNX)	-	文本转语音
OCR	Tesseract	-	图片文字识别

3.2 前端技术栈

组件	技术选型	版本	用途
框架	Next.js	15	React全栈框架
UI库	shadcn/ui + Tailwind CSS	-	组件库和样式
状态管理	React Context + useReducer	-	全局状态管理
数据库ORM	Drizzle ORM	-	PostgreSQL操作
HTTP客户端	Fetch API (原生)	-	API请求

3.3 AI模型

模型	用途	部署方式
Lusizo/qwen2.5-7b-instruct-1m	LLM推理（查询重写、IRAC生成）	Ollama本地
Qwen/Qwen3-Embedding-0.6B	文本向量化	Sentence Transformers
BAAI/bge-reranker-v2-m3	检索结果重排序	CrossEncoder

4. 核心模块详解

4.1 配置管理 (src/config.py)

集中管理所有系统配置参数：

```
class Config:
    # 向量存储路径
    DB_PATH = "data/vector_store" # 存放 embeddings.npy 和 metadata.json

    # AI模型配置
    SEARCH_MODEL = "Qwen/Qwen3-Embedding-0.6B" # Embedding模型
    RERANK_MODEL = "BAAI/bge-reranker-v2-m3" # Reranker模型
    RAG_MODEL = "Lusizo/qwen2.5-7b-instruct-1m:latest" # LLM模型

    # 检索参数
    DEFAULT_N_RESULTS = 15 # 向量海选数量
    DEFAULT_TOP_N = 5 # 最终引用数量
    DEFAULT_THRESHOLD = -2 # 相关性阈值
    DEFAULT_MAX_LENGTH = 512 # Reranker最大token长度

    # HuggingFace配置
    HF_HUB_OFFLINE = 1 # 离线模式，避免SSL错误
```

4.2 查询重写模块 (src/RAG.py)

4.2.1 功能概述

将用户的自然语言问题转换为结构化的检索指令，提升检索精度。

4.2.2 核心函数

```
rewrite_query(user_query, history, model_name)
```

- **输入:** 用户问题、对话历史、模型名称
- **输出:** 结构化搜索指令字符串
- **格式:** 领域:XXX | 【法律名-第xxx条】 | 法律关键词:A、B、C | 案例关键词:D、E、F

处理流程:

用户问题

|

▼

构建Rewrite Prompt

- 注入可用法律清单 (registry.json)
- 注入对话历史
- 包含Few-shot示例

|

▼

调用 Ollama LLM

- num_ctx=4096
- temperature=0.3

|

▼

解析输出

- 提取领域、法律名、关键词
- 失败时回退到原始查询

Prompt设计要点:

- 提供 data/registry.json 中的完整法律/案例清单
- 包含丰富的Few-shot示例
- 明确的红线规则（禁止编造法律名称）
- 支持法律条文和案例号的精准识别

4.2.3 IRAC生成

```
call_ollama_rag(query_text, retrieved_docs, history, model_name)
```

- 有法律依据时: 使用IRAC框架Prompt
- 无法律依据时: 简单对话Prompt

IRAC Prompt结构:

你是一位专业的中国法律分析师...

检索到的法律依据

{法律/解释文档}

检索到的案例

{案例文档}

分析要求

请按照IRAC框架进行分析：

1. Issue（争议焦点）：识别核心法律问题
2. Rule（法律规则）：引用相关法律条文
3. Application（法律适用）：将规则应用于具体事实
4. Conclusion（结论）：给出法律意见和风险提示

4.3 混合检索模块 (src/search.py)

4.3.1 检索策略

系统采用三路混合检索策略，确保检索的准确性和全面性：

用户问题（经查询重写）



第一路：精准标签匹配

解析 【法律名-第xxx条】 或 【案例号】

在metadata中精确匹配

匹配结果标记 method="精准点名"



第二路：分类型向量搜索

法律关键词

→ 向量搜索

→ doc_type=law_interp

→ method="法律检索"

案例关键词

→ 向量搜索

→ doc_type=case

→ method="案例检索"



第三路：关系扩展

从 reverse_references.json 读取关联索引

- 命中法律但无案例 → 补充关联案例

- 命中案例但无法律 → 补充关联法律/解释



合并去重结果

4.3.2 向量搜索实现

```
_vector_search(query_text, n_results, doc_type_filter)
```

核心算法

1. 使用SentenceTransformer将查询文本向量化
2. 计算与所有文档向量的余弦相似度
3. 使用np.argpartition高效取top-k
4. 按source_article_number去重

性能优化:

- 向量预加载为float32 numpy数组
- 使用 np.argpartition 而非全排序
- 懒加载全局单例模式

4.3.3 精准标签解析

```
parse_precise_tag(tag_text)
```

支持的格式:

- 刑法-第十条
- 刑法 第十条
- 刑法第十条
- 指导案例1号
- (2023) 最高法民申1234号
- 案例- 张某故意杀人案

4.4 重排序模块 (src/rerank.py)

4.4.1 两轮重排序策略

第一轮: 相关性评分

检索结果列表

|

▼

CrossEncoder 评分

- 对每个 (query, content) 对打分
- 精准点名文档 → score=999 (VIP)
- 过滤 `score < threshold` 的文档

|

▼

评分结果列表

第二轮：类型均衡选择

```
_type_balanced_select(scored_docs, top_n)
```

评分结果列表

|

▼

分离VIP文档 (score>=999)

|

▼

普通文档按类型分组

- law_docs: 法律/解释
- case_docs: 案例

|

▼

均衡选择策略

1. 至少1条法律/解释
2. 至少1条案例
3. 剩余名额按分数填充

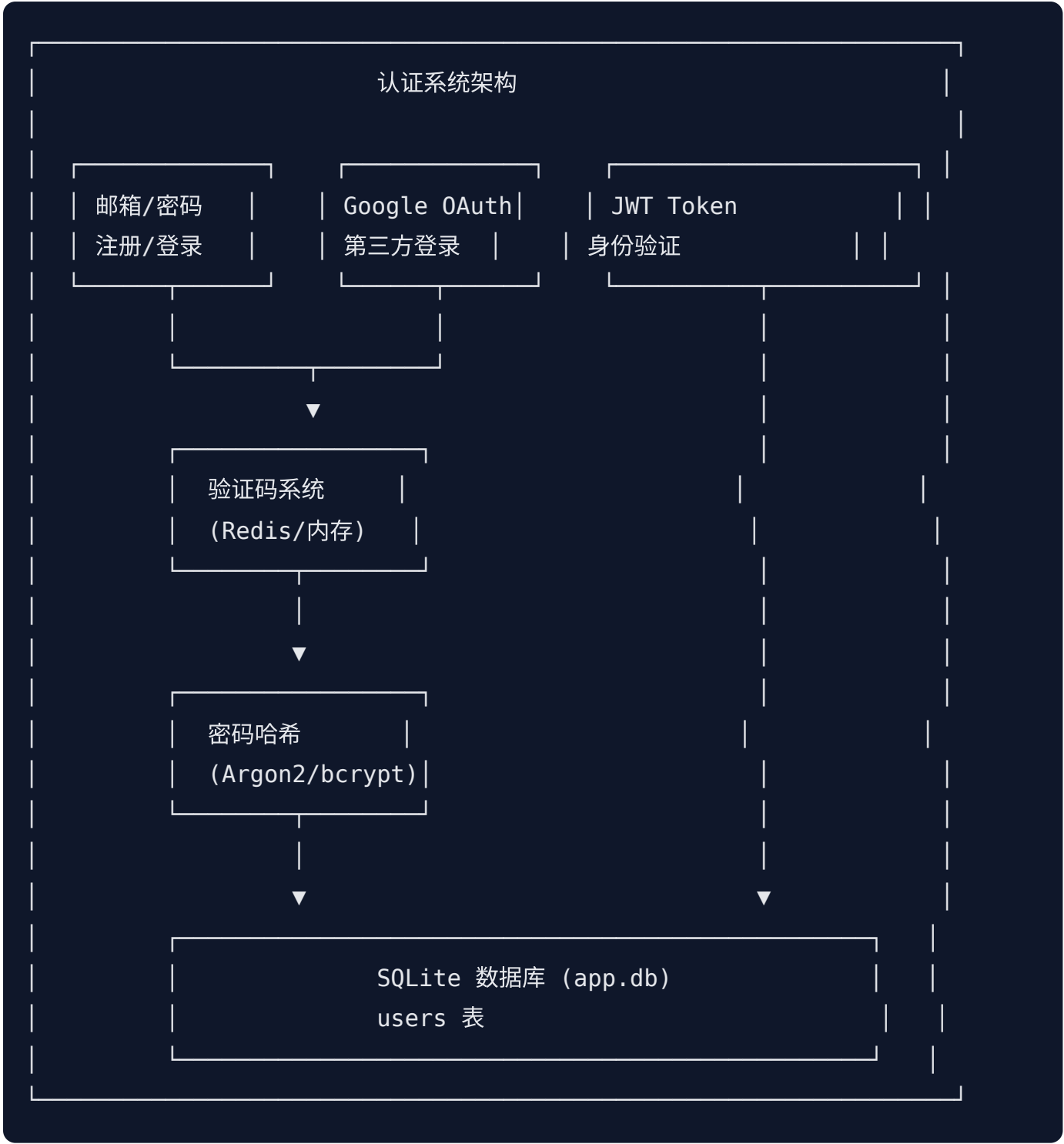
|

▼

最终top_n结果

4.5 认证系统

4.5.1 认证架构



4.5.2 认证流程

邮箱注册流程:

1. 用户提交邮箱 → POST /api/auth/send-code

- |
- ├─ 检查邮箱未注册
- ├─ 60秒防抖检查
- ├─ 生成6位验证码 (secrets.randbelow)
- ├─ HMAC-SHA256摘要存储
- └─ 异步发送验证邮件

2. 用户提交注册信息 → POST /api/auth/register

- |
- ├─ 验证验证码
- ├─ Argon2哈希密码
- ├─ 创建用户 (默认tier="premium")
- ├─ 解析管理员角色 (ADMIN_EMAILS/INVITE_CODE)
- └─ 发送欢迎邮件

3. 用户登录 → POST /api/auth/login

- |
- ├─ 查找用户
- ├─ 验证密码哈希
- └─ 生成JWT (HS256, 120分钟有效期)

Google OAuth流程:

1. 前端获取Google ID Token

|

▼

2. POST /api/auth/google {credential}

- |
- ├─ 验证Google Token (google.oauth2.id_token)
- ├─ 检查audience/issuer
- ├─ 查找或创建用户
- └─ 生成JWT

4.5.3 JWT令牌结构


```
{
  "sub": "user-uuid",
  "email": "user@example.com",
  "role": "user|admin",
  "exp": 1234567890
}
```

- 签名算法: HS256
- 有效期: 120分钟
- 存储: HttpOnly Cookie + sessionStorage

4.6 文档解析模块 (src/data_process.py)

4.6.1 法律原文解析

```
parse_code_perfect(file_path, output_json)
```

解析层级结构:

```
编 (第一编 总则)
├── 分编 (第一分编)
│   ├── 章 (第一章)
│   │   ├── 节 (第一节)
│   │   │   └── 条 (第一条)
```

输出格式:

```
{
  "id": "law_criminal_article_1",
  "article_number": "第一条",
  "hierarchy": {
    "编": "第一编 总则",
    "章": "第一章 刑法的任务、基本原则和适用范围"
  },
  "content": "为了惩罚犯罪，保护人民...",
  "source": "中华人民共和国刑法",
  "doc_type": "law"
}
```

4.6.2 司法解释解析

```
parse_interpretation_perfect(file_path, output_json)
```

层级简化:

```
章（一、关于一般规定）
└─ 条（第一条）
```

4.6.3 案例解析

```
parse_case(file_path, output_json)
```

处理策略:

- 短文档(<2000字): 合并为一条记录
- 长文档: 按段落结构拆分（本院认为、经审理查明等）

5. 数据流与处理流程

5.1 完整请求处理流程

用户在前端输入法律问题



前端处理 (Next.js)

1. 构建请求体 {query, history, top_n, n_results, stream}
2. 添加JWT Authorization头
3. POST到 /api/chat



非法律问题检测

- 关键词启发式判断
- 匹配非法律关键词且不含法律关键词 → 普通对话



▼ (法律问题)

阶段1: 查询重写 (RAG.py:rewrite_query)

- Ollama生成结构化搜索指令
- 输出: 领域 | 法律标签 | 法律关键词 | 案例关键词



阶段2: 混合检索 (search.py:run_search)

1. 精准标签匹配
2. 法律/解释向量搜索
3. 案例向量搜索
4. 关系扩展



阶段3: 重排序 (rerank.py:rerank_context)

1. CrossEncoder评分

- 2. 类型均衡选择
- 3. 输出top_n结果



无搜索结果
→ 普通对话

有搜索结果
→ IRAC分析

阶段4: IRAC生成 (api_server.py)

- 1. 分离法律/解释和案例
- 2. 构建IRAC Prompt
- 3. Ollama生成JSON格式分析
- 4. 解析/降级处理

响应组装

```
{
  "answer": "IRAC Markdown文本",
  "irac": {issue, rule, application, conclusion},
  "references": [{source, article, content, type}],
  "metadata": {query_rewrite, search_method, ...}
}
```

SSE流式响应

- 每4字符发送 delta 事件
- 最后发送 done 事件 (包含完整references和irac)

↓

▼

前端渲染

- readChatResponse 解析SSE流
- IracDisplay 渲染IRAC四部分
- LegalReferenceCard 渲染溯源卡片

5.2 离线数据处理流程

法律/案例 .docx 文件

↓

▼

scripts/create_registry.py

扫描 data/法律原文/, data/司法解释/, data/案例/

生成 data/registry.json

↓

▼

scripts/process_and_ingest.py

1. 读取 registry.json
2. 调用 data_process.py 解析 .docx
3. 调用 ingest.py 向量化
4. 保存 embeddings.npy + metadata.json

↓

▼

data/vector_store/

- └─ embeddings.npy (向量矩阵)
- └─ metadata.json (文档元数据)

6. API接口规范

6.1 认证接口

POST /api/auth/send-code

功能: 发送邮箱验证码

请求体:

```
{
  "email": "user@example.com",
  "purpose": "register" // 或 "reset"
}
```

响应:

```
{
  "status": "success",
  "data": null,
  "msg": "验证码已发送"
}
```

业务规则:

- 注册: 检查邮箱未注册
- 重置: 检查邮箱已注册
- 60秒防抖

POST /api/auth/register

功能: 用户注册

请求体:

```
{
  "email": "user@example.com",
  "password": "securePassword123",
  "full_name": "张三",
  "code": "123456",
  "invite_code": "optional_admin_code"
}
```

响应:

```
{
  "status": "success",
  "data": {
    "user_id": "uuid",
    "email": "user@example.com",
    "role": "user"
  },
  "msg": "注册成功"
}
```

POST /api/auth/login

功能: 用户登录

请求体:

```
{
  "email": "user@example.com",
  "password": "securePassword123"
}
```

响应:

```
{
  "status": "success",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIs... ",
    "user": {
      "id": "uuid",
      "email": "user@example.com",
      "username": "张三",
      "role": "user"
    }
  },
  "msg": "登录成功"
}
```

POST /api/auth/google

功能: Google OAuth登录

请求体:

```
{
  "credential": "google_id_token"
}
```

POST /api/auth/reset-password

功能: 重置密码

请求体:

```
{
  "email": "user@example.com",
  "code": "123456",
  "new_password": "newSecurePassword123"
}
```

POST /api/auth/token/verify

功能: 验证JWT有效性

请求体:

```
{
  "token": "eyJhbGciOiJIUzI1NiIs..."
}
```

6.2 核心对话接口

POST /api/chat

功能: 法律问答主接口

请求头:

```
Authorization: Bearer <jwt_token>
Content-Type: application/json
```

请求体:

```
{
  "query": "故意杀人罪的量刑标准是什么？",
  "history": [
    {"role": "user", "content": "之前的问题"},
    {"role": "assistant", "content": "之前的回答"}
  ],
  "top_n": 5,
  "n_results": 15,
  "threshold": -2,
  "force_search": true,
  "stream": false
}
```

参数说明:

参数	类型	默认值	说明
query	string	必填	用户问题
history	array	[]	对话历史
top_n	int	5	最终引用文档数量
n_results	int	15	向量海选数量
threshold	float	-2	相关性阈值
force_search	bool	true	强制搜索
stream	bool	false	是否流式响应

JSON响应:

```
{
  "status": "success",
  "data": {
    "answer": "## Issue (争议焦点) \n...",
    "irac": {
      "issue": "故意杀人罪的量刑标准问题",
      "rule": "《刑法》第二百三十二条规定...",
      "application": "根据案件事实...",
      "conclusion": "综上所述..."
    },
    "references": [
      {
        "source": "中华人民共和国刑法",
        "article_number": "第二百三十二条",
        "content": "故意杀人的，处死刑、无期徒刑或者十年以上有期徒刑...",
        "doc_type": "law",
        "score": 0.95
      }
    ],
    "metadata": {
      "query_rewrite": "领域:刑事|【刑法-第二百三十二条】|...",
      "search_methods": ["精准点名", "法律检索"],
      "total_candidates": 15,
      "final_count": 5
    }
  },
  "msg": null
}
```

SSE流式响应格式:

```
data: {"type": "delta", "content": "##"}
```

```
data: {"type": "delta", "content": " Issue"}
```

```
data: {"type": "delta", "content": " (争议"}
```

```
...
```

```
data: {"type": "done", "references": [...], "irac": {...}}
```

6.3 文件上传接口

POST /api/upload

功能: 文件上传（支持图片OCR和文档解析）

请求头:

```
Authorization: Bearer <jwt_token>  
Content-Type: multipart/form-data
```

请求体:

```
file: <binary>
```

支持格式:

- 图片: jpg, png, gif, webp (最大10MB)
- 文档: pdf, doc, docx, txt, md (最大10MB)

响应:

```
{
  "status": "success",
  "data": {
    "filename": "document.pdf",
    "extracted_text": "提取的文本内容...",
    "file_type": "pdf",
    "file_size": 1024000
  },
  "msg": "文件上传成功"
}
```

6.4 语音接口

POST /api/speech/synthesize

功能: 文本转语音

请求体:

```
{
  "text": "要转换的文本",
  "character": "nina" // 或 "feibi"
}
```

响应: WAV音频流 (32kHz采样率)

6.5 在线状态接口

POST /api/presence/heartbeat

功能: 用户心跳上报

请求头:

```
Authorization: Bearer <jwt_token>
```

GET /api/presence/online-count

功能: 获取在线用户数

响应:

```
{
  "status": "success",
  "data": {"count": 42},
  "msg": null
}
```

7. 数据库设计

7.1 后端SQLite数据库 (`app.db`)

users表

```
CREATE TABLE users (
  id VARCHAR(36) PRIMARY KEY,          -- UUID主键
  email VARCHAR(255) UNIQUE NOT NULL,  -- 邮箱 (唯一索引)
  username VARCHAR(100) NOT NULL,      -- 用户名
  password_hash VARCHAR(255),          -- 密码哈希 (Google用户可为
NULL)
  provider VARCHAR(32) NOT NULL DEFAULT 'local', -- 认证提供者
  google_id VARCHAR(255) UNIQUE,        -- Google用户ID (唯一)
  avatar_url VARCHAR(500),              -- 头像URL
  role VARCHAR(32) NOT NULL DEFAULT 'user', -- 角色: user/admin
  tier VARCHAR(32) NOT NULL DEFAULT 'free', -- 层级: free/premium
  is_premium BOOLEAN NOT NULL DEFAULT 1, -- 是否高级用户
  is_verified BOOLEAN NOT NULL DEFAULT 0, -- 是否已验证邮箱
  created_at DATETIME,                  -- 创建时间
  updated_at DATETIME                    -- 更新时间
);

CREATE INDEX ix_users_email ON users(email);
```

verification_codes表

```
CREATE TABLE verification_codes (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    email VARCHAR(255) NOT NULL,  
    code VARCHAR(6) NOT NULL,  
    expires_at DATETIME NOT NULL  
);  
  
CREATE INDEX ix_verification_codes_email ON verification_codes(email);
```

注意: 验证码实际存储在Redis或内存缓存中，此表为早期遗留。

7.2 向量存储

embeddings.npy

- **格式:** numpy数组
- **形状:** (N, D)，N为文档数量，D为向量维度
- **数据类型:** float32

metadata.json

```
[  
  {  
    "id": "law_criminal_article_1",  
    "article_number": "第一条",  
    "source": "中华人民共和国刑法",  
    "doc_type": "law",  
    "content": "为了惩罚犯罪，保护人民...",  
    "hierarchy": {  
      "编": "第一编 总则",  
      "章": "第一章..."  
    }  
  },  
  ...  
]
```

7.3 辅助数据文件

文件	用途
data/registry.json	法律/案例清单，用于prompt注入
data/reverse_references.json	法律-案例关联索引
data/all_records.json	全量记录，用于关系扩展

7.4 前端PostgreSQL数据库 (Drizzle ORM)

注意: 前端数据库用于原chatbot功能，法律助手的对话历史仅存储在前端localStorage中。

表名	主要字段	用途
User	id, email, password, fullName, role	用户信息
Chat	id, userId, title, visibility	聊天会话
Message_v2	id, chatId, role, parts(JSON)	消息记录
Vote_v2	chatId, messageId, isUpvoted	消息投票
Document	id, userId, title, content, kind	文档管理
Stream	id, chatId	流式会话

8. 前端架构

8.1 目录结构


```

next-app/
├─ app/
│   ├─ (auth)/                # 认证页面组
│   │   ├─ login/page.tsx     # 登录页
│   │   ├─ register/page.tsx  # 注册页
│   │   └─ forgot-password/    # 忘记密码
│   ├─ api/
│   │   └─ auth/[...proxy]/    # 认证代理路由
│   ├─ legal-assistant/
│   │   └─ page.tsx            # 法律助手主页面
│   └─ layout.tsx              # 根布局
├─ components/
│   ├─ legal-assistant/        # 法律助手核心组件
│   │   ├─ chat.tsx            # 主聊天组件
│   │   ├─ message-bubble.tsx  # 消息气泡
│   │   ├─ irac-display.tsx    # IRAC展示组件
│   │   ├─ reference-card.tsx  # 溯源卡片
│   │   └─ ...                  # 80+组件文件
│   ├─ auth/                    # 认证组件
│   │   └─ google-login.tsx    # Google登录按钮
│   └─ ui/                      # shadcn/ui基础组件
├─ lib/
│   ├─ api/
│   │   └─ auth.ts              # 认证API封装
│   ├─ auth/
│   │   ├─ session-client.ts    # 客户端session管理
│   │   └─ explicit-auth-proxy.ts # Next.js认证代理
│   ├─ api-url.ts              # API URL路由策略
│   └─ db/                      # Drizzle ORM配置
└─ ...

```

8.2 认证流程

前端认证流程

1. 登录页提交表单

|

▼

2. 调用 lib/api/auth.ts 的 login()

|

▼

3. POST /api/auth/login (Next.js代理)

|

▼

4. 代理转发到后端 → 获取JWT

|

▼

5. 设置 legal_auth_token HttpOnly Cookie
同时存入 sessionStorage/localStorage

|

▼

6. 跳转到 /legal-assistant

|

▼

7. validateStoredAuthSession() 验证token

|

▼

8. 后续请求携带 Authorization: Bearer {token}

8.3 API路由策略

```
// lib/api-url.ts
export function getApiUrl(path: string): string {
  if (path.startsWith('/api/auth/')) {
    // 认证接口走Next.js代理（同源）
    return path;
  }
  // 其他接口直接请求后端（跨域）
  return `${process.env.NEXT_PUBLIC_API_BASE_URL}${path}`;
}
```

8.4 状态管理

使用React Context + useReducer模式：

```
// 核心状态结构
interface ChatState {
  messages: Message[];
  isLoading: boolean;
  currentIrac: IracData | null;
  references: Reference[];
}

// Action类型
type ChatAction =
  | { type: 'ADD_MESSAGE'; payload: Message }
  | { type: 'UPDATE_MESSAGE'; payload: { id: string; content: string } }
  | { type: 'FINALIZE_MESSAGE'; payload: { references: Reference[]; irac: IracData } }
  | { type: 'SET_LOADING'; payload: boolean };
```

8.5 核心组件

IracDisplay组件

渲染IRAC四部分，支持折叠/展开：

```
<IracDisplay irac={iracData} />
// 渲染为可折叠卡片：
// - Issue（争议焦点）
// - Rule（法律规则）
// - Application（法律适用）
// - Conclusion（结论）
```

LegalReferenceCard组件

渲染溯源卡片，支持全文查看和引用复制：

```
<LegalReferenceCard references={references} />
// 渲染为卡片列表，每张卡片显示：
// - 法律来源
// - 条文编号
// - 内容摘要
// - 展开全文按钮
// - 复制引用按钮
```

9. 部署与运维

9.1 环境要求

后端

- Python 3.13+
- CUDA (可选，用于GPU加速)
- 内存: 8GB+ (模型加载)
- 磁盘: 10GB+ (模型和数据)

前端

- Node.js 18+
- pnpm

9.2 安装步骤

后端安装

```
# 1. 创建conda环境
conda create -n RAG-Legal python=3.13
conda activate RAG-Legal

# 2. 安装依赖
pip install -r requirements.txt

# 3. 配置环境变量
cp .env.example .env
# 编辑 .env 填入SMTP、Google OAuth等配置

# 4. 下载Ollama模型
ollama pull Lusizo/qwen2.5-7b-instruct-1m

# 5. 初始化数据（如有新数据）
python scripts/create_registry.py
python scripts/process_and_ingest.py

# 6. 启动后端
python src/api_server.py
```

前端安装

```
cd next-app

# 1. 安装依赖
pnpm install

# 2. 配置环境变量
# 创建 .env.local 文件
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000

# 3. 启动开发服务器
pnpm dev

# 4. 构建生产版本
pnpm build
pnpm start
```

9.3 环境变量配置

后端 .env

```
# SMTP邮件配置
SMTP_HOST=smtp.example.com
SMTP_PORT=465
SMTP_USER=your-email@example.com
SMTP_PASSWORD=your-password
SMTP_FROM=noreply@example.com

# Google OAuth
GOOGLE_CLIENT_ID=your-google-client-id

# 管理员配置
ADMIN_EMAILS=admin1@example.com,admin2@example.com
ADMIN_INVITE_CODE=your-admin-invite-code

# Redis (可选)
REDIS_URL=redis://localhost:6379

# HuggingFace镜像
HF_ENDPOINT=https://hf-mirror.com
```

前端 .env.local

```
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000
DATABASE_URL=postgresql://user:password@localhost:5432/dbname
```

9.4 服务端口

服务	端口	说明
Next.js前端	3000	Web界面
FastAPI后端	8000	API服务
Ollama	11434	LLM服务
genie_tts	9900	TTS服务

9.5 性能优化建议

1. **GPU加速**: 安装CUDA版本的PyTorch，自动启用GPU加速
 2. **模型预热**: 启动时预加载模型，避免首次请求延迟
 3. **向量索引**: 对于大规模数据，考虑使用FAISS替代numpy
 4. **缓存策略**: 启用Redis缓存验证码和热点查询结果
 5. **CDN加速**: 前端静态资源使用CDN分发
-

10. 关键设计决策

10.1 双数据库架构

决策: 后端使用SQLite存储用户/认证数据，前端使用PostgreSQL存储聊天历史/文档。

原因:

- SQLite轻量级，适合单机部署的认证系统
- PostgreSQL支持原chatbot-ui的复杂查询需求
- 法律助手的对话历史仅在前端localStorage中，无需后端持久化

10.2 自建向量存储

决策: 使用numpy矩阵（embeddings.npy + metadata.json）而非ChromaDB。

原因:

- 数据规模较小（约3000条），numpy足够高效
- 避免ChromaDB的额外依赖和复杂性
- 便于调试和理解底层实现

10.3 三路混合检索

决策: 标签精准匹配 + 分类型向量搜索 + 关系扩展。

原因:

- 标签匹配: 支持用户精准引用法律条文
- 分类型搜索: 确保法律和案例两类文档都有代表性
- 关系扩展: 补充关联的法律/案例，提升回答完整性

10.4 IRAC框架

决策: 要求LLM输出结构化的IRAC JSON分析。

原因:

- IRAC是法律分析的标准框架，符合法律专业人士的思维模式
- JSON格式便于前端结构化渲染
- 降级策略确保系统鲁棒性

10.5 认证代理模式

决策: 认证接口通过Next.js代理转发，核心接口直接跨域请求。

原因:

- 认证接口需要设置HttpOnly Cookie，必须同源
- 核心接口直接请求后端，减少代理延迟
- CORS白名单控制跨域访问安全

10.6 SSE流式响应

决策: 使用Server-Sent Events实现流式响应，每4字符发送一次delta。

原因:

- 提升用户体验，实时显示生成过程
- 4字符粒度平衡了实时性和网络开销
- done事件携带完整references和irac，确保数据完整性

附录A: 文件清单

核心业务代码

文件	功能
src/api_server.py	FastAPI入口，核心对话接口
src/RAG.py	查询重写，IRAC提示词，LLM调用
src/search.py	混合检索实现
src/rerank.py	交叉编码器重排序
src/config.py	配置管理

src/database.py	数据库连接和初始化
src/models.py	数据模型定义
src/utils.py	工具函数（认证核心）
src/injest.py	向量化入库
src/data_process.py	文档解析
src/routers/auth.py	认证路由
src/routers/multimodal.py	多模态路由
src/api_voice_genie.py	TTS语音服务

脚本工具

文件	功能
scripts/create_registry.py	生成registry.json
scripts/process_and_ingest.py	解析+向量化入库
scripts/build_vector_store.py	构建向量库
scripts/generate_cases.py	生成案例文件
scripts/main_cli.py	CLI测试入口
scripts/debug_auth_flow.py	认证调试脚本

数据文件

文件/目录	说明
data/法律原文/	26部法律.docx文件
data/司法解释/	9份司法解释.docx文件
data/案例/	200+案例.docx文件
data/vector_store/	向量库存储目录

data/registry.json	法律/案例清单
data/reverse_references.json	关联索引

附录B: 术语表

术语	说明
RAG	Retrieval-Augmented Generation，检索增强生成
IRAC	Issue-Rule-Application-Conclusion，法律分析框架
Embedding	文本向量化，将文本转换为数值向量
CrossEncoder	交叉编码器，用于计算query-document相关性分数
SSE	Server-Sent Events，服务端推送事件
JWT	JSON Web Token，身份认证令牌
OCR	Optical Character Recognition，光学字符识别
TTS	Text-to-Speech，文本转语音
STT	Speech-to-Text，语音转文本

文档结束

本说明书基于代码库版本 2026-06-24 生成，如有更新请同步修改。